

```

--viewing the table
SELECT *
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
LIMIT 100;

--product detail starts with 'Dark'
SELECT transaction_id,product_detail, unit_price
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail LIKE 'Dark%'
LIMIT 10;

--The number of products that starts with 'Dark' are 3
SELECT DISTINCT product_detail
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail LIKE 'Dark%';

--The word that contain 'Chai'
SELECT DISTINCT product_type, product_detail
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail LIKE '%Chai%'
ORDER BY product_type;

--The product details ends with 'Lg'
SELECT transaction_id,product_detail, product_category, unit_price
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail LIKE '%Lg'
ORDER BY unit_price DESC;

--Use NOT LIKE to find all products that are NOT a type of tea
SELECT DISTINCT product_category, product_type,product_detail
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_category NOT LIKE 'Tea'
ORDER BY product_category, product_type;

--Find products with two characters
SELECT DISTINCT product_detail
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail LIKE '%__Sm'
OR product_detail LIKE '%__Rg'
OR product_detail LIKE '%__Lg'
ORDER BY product_detail;

--Use RLIKE to find all products where the product_detail contains a number

```

```

SELECT DISTINCT product_detail
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail RLIKE '[a-zA-Z]'
ORDER BY product_detail;

--Combine two wildcards
SELECT transaction_id,transaction_date,store_location,product_detail,unit_price
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_category = 'Coffee'
AND(product_detail LIKE '%Brazilian%' OR product_detail LIKE '%Ethiopian%')
ORDER BY transaction_date;

-- revenue per store
SELECT
    store_location,
    ROUND(SUM(transaction_qty * unit_price), 2) AS store_total_revenue
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
GROUP BY store_location
LIMIT 20;

-- transaction shows what percentage of its store's total revenue that transaction
represents.
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROUND(
        ((transaction_qty * unit_price) * 100.0) /
        SUM(transaction_qty * unit_price) OVER (PARTITION BY store_location),
        2
    ) AS pct_of_store_revenue
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
LIMIT 20;

--running total of revenue per store, ordered by transaction date and time.
SELECT
    transaction_id,
    store_location,
    transaction_date,
    transaction_time,
    ROUND(transaction_qty * unit_price, 2) AS revenue,

```

```

ROUND(
    SUM(transaction_qty * unit_price) OVER (
        PARTITION BY store_location
        ORDER BY transaction_date, transaction_time
    ),
    2
) AS running_total
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
LIMIT 30;

--Rank every transaction within its store by revenue (highest first).
SELECT
    transaction_id,
    store_location,
    ROUND((transaction_qty * unit_price), 2) AS revenue,
    ROW_NUMBER()OVER( PARTITION BY store_location ORDER BY transaction_qty*
unit_price DESC) AS row_num
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
ORDER BY store_location, row_num, unit_price DESC
LIMIT 30;

--Compare RANK, DENSE_RANK, and ROW_NUMBER side by side on the same data.
SELECT
    product_category,
    product_detail,
    unit_price,
    ROW_NUMBER() OVER( PARTITION BY product_category ORDER BY unit_price DESC) AS
rn,
    RANK() OVER(PARTITION BY product_category ORDER BY unit_price DESC) AS rnk,
    DENSE_RANK() OVER( PARTITION BY product_category ORDER BY unit_price DESC) AS
d_rnk
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
ORDER BY product_category, unit_price DESC
LIMIT 40;

--Use NTILE to divide transactions into 4 revenue quartiles within each store.
SELECT
    transaction_id,
    store_location,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    NTILE(4) OVER (PARTITION BY store_location ORDER BY transaction_qty*unit_price
ASC) AS quartile
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`

```

```
ORDER BY store_location, revenue ASC
LIMIT 30;
```

--Use LAG to compare each transaction's revenue to the previous transaction in the same store.

```
SELECT
    transaction_id,
    store_location,
    transaction_date,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROUND( LAG(transaction_qty*unit_price, 1, 0) OVER(
        ORDER BY transaction_date ), 2) AS prev_revenue
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
LIMIT 20;
```

--Use LEAD to show the next transaction's revenue alongside the current one.

```
SELECT
    transaction_id,
    store_location,
    transaction_date,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    ROUND( LEAD(transaction_qty*unit_price, 1) OVER( ORDER BY transaction_date),
2) AS next_revenue
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
LIMIT 20;
```

--Use FIRST_VALUE to show the cheapest product in each category alongside every row.

```
SELECT DISTINCT
    product_category,
    product_detail,
    unit_price,
    FIRST_VALUE(product_detail) OVER( PARTITION BY product_category ORDER BY
unit_price ASC) AS cheapest_in_category
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
ORDER BY product_category, unit_price
LIMIT 30;
```

--use DISTINCT to find the cheapest category

```
SELECT DISTINCT
    product_category,
    FIRST_VALUE(product_detail) OVER(
        PARTITION BY product_category
```

```

        ORDER BY unit_price ASC
    ) AS cheapest_in_category
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
ORDER BY product_category
LIMIT 30;

-- combines wildcards AND window functions
SELECT
    transaction_id,
    store_location,
    product_detail,
    ROUND(transaction_qty * unit_price, 2) AS revenue,
    RANK() OVER(PARTITION BY store_location ORDER BY transaction_qty*unit_price
DESC) AS store_rank,
    ROUND(AVG(transaction_qty*unit_price) OVER(PARTITION BY store_location), 2) AS
store_avg_revenue
FROM `casestudypart1`.`default`.`bright_coffee_shop_analysis_case_study_1`
WHERE product_detail LIKE '%Lg'
OR product_detail LIKE '%Rg'
ORDER BY store_location, store_rank
LIMIT 40;

```